

Understanding JWT

What is JWT?

- JWT stands for JSON Web Token
- Internet standard ([IETF 7519 Link](#)) for creating [JSON](#)-based [access tokens](#)
- JWT can be typically used to pass identity of authenticated users and [service provider](#),

JWT Process Flow ([diagram link](#))

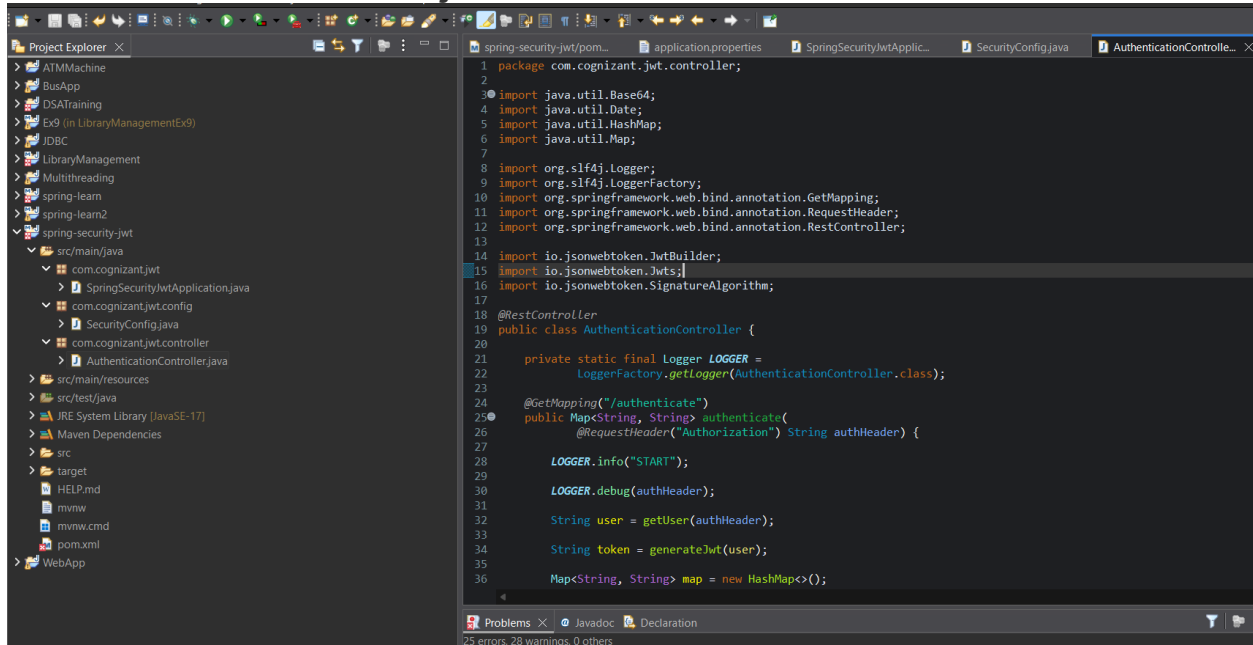
- Client sends username and password to server
- Servers validates credentials, creates token (JWT) and reponds it back
- Client attaches the token in the subsequent requests to server
- Server validates the token (JWT) on each client request

Structure of JSON Web Token

- Reference: https://en.wikipedia.org/wiki/JSON_Web_Token#Structure
- Header: Contains the encryption algorithm
- Payload: Contains application specific data. Usually this contains the user id and role.
- Signature: Computed based on the formula defined using header and payload

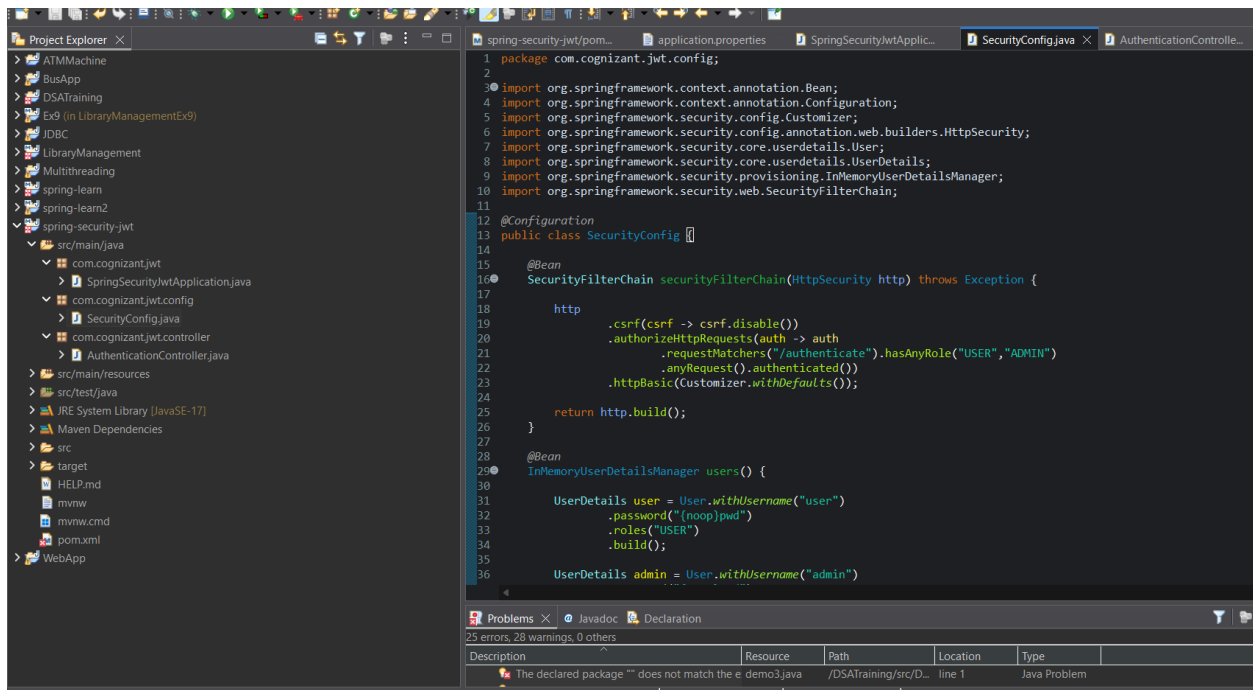
Create authentication controller and configure it in SecurityConfig

AuthenticationController.java



```
1 package com.cognizant.jwt.controller;
2
3 import java.util.Base64;
4 import java.util.Date;
5 import java.util.HashMap;
6 import java.util.Map;
7
8 import org.slf4j.Logger;
9 import org.slf4j.LoggerFactory;
10 import org.springframework.web.bind.annotation.GetMapping;
11 import org.springframework.web.bind.annotation.RequestHeader;
12 import org.springframework.web.bind.annotation.RestController;
13
14 import io.jsonwebtoken.JwtBuilder;
15 import io.jsonwebtoken.Jwts;
16 import io.jsonwebtoken.SignatureAlgorithm;
17
18 @RestController
19 public class AuthenticationController {
20
21     private static final Logger LOGGER =
22         LoggerFactory.getLogger(AuthenticationController.class);
23
24     @GetMapping("/authenticate")
25     public Map<String, String> authenticate(
26         @RequestHeader("Authorization") String authHeader) {
27
28         LOGGER.info("START");
29
30         LOGGER.debug(authHeader);
31
32         String user = getUser(authHeader);
33
34         String token = generateJwt(user);
35
36         Map<String, String> map = new HashMap<>();
```

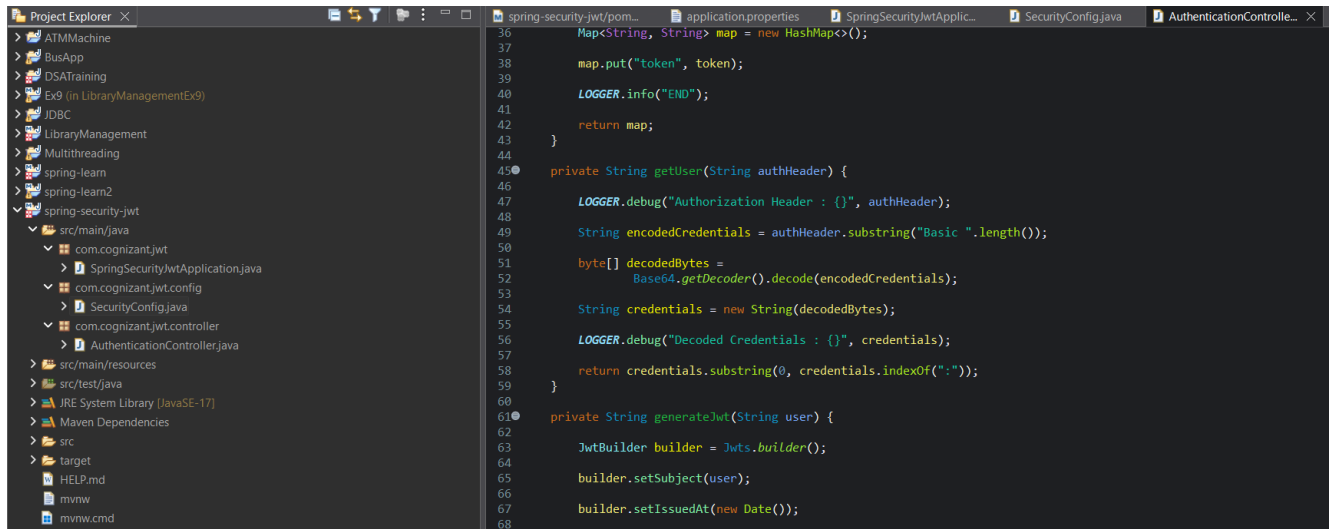
SecurityConfig.java



```
1 package com.cognizant.jwt.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.Customizer;
6 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 import org.springframework.security.core.userdetails.User;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.security.provisioning.InMemoryUserDetailsManager;
10 import org.springframework.security.web.SecurityFilterChain;
11
12 @Configuration
13 public class SecurityConfig {
14
15     @Bean
16     SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
17
18         http
19             .csrf(csrf -> csrf.disable())
20             .authorizeHttpRequests(auth -> auth
21                 .requestMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
22                 .anyRequest().authenticated())
23             .httpBasic(Customizer.withDefaults());
24
25         return http.build();
26     }
27
28     @Bean
29     InMemoryUserDetailsManager users() {
30
31         UserDetails user = User.withUsername("user")
32             .password("{noop}pwd")
33             .roles("USER")
34             .build();
35
36         UserDetails admin = User.withUsername("admin")
```

Description	Resource	Path	Location	Type
The declared package "" does not match the e demo3.java	/D/SA/Training/src/D...	line 1		Java Problem

Read Authorization header and decode the username and password



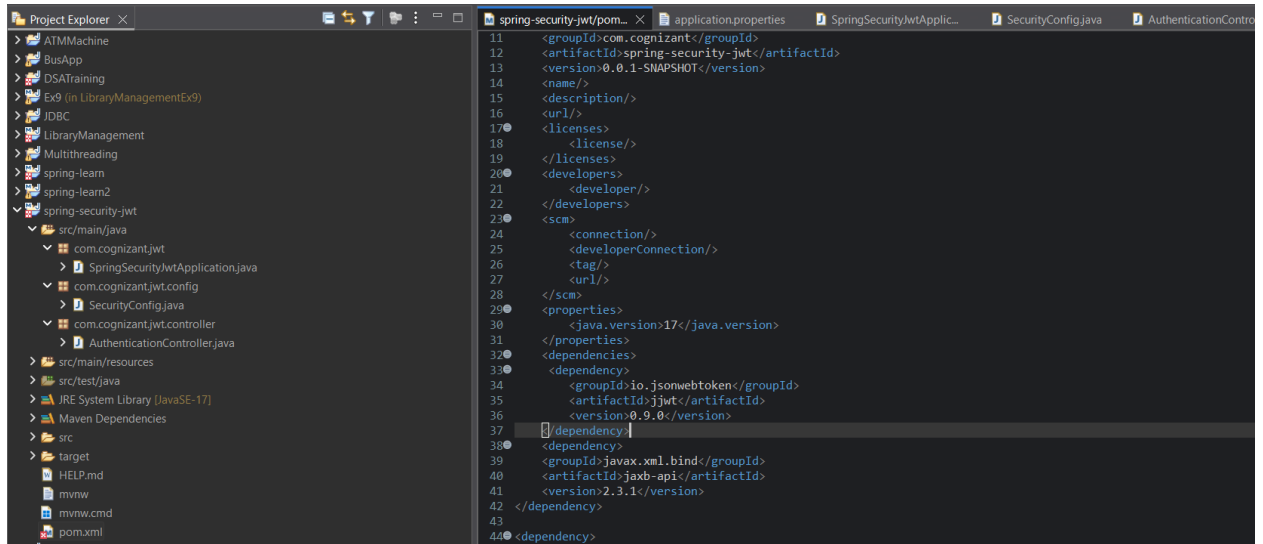
The screenshot shows an IDE with a project explorer on the left and a code editor on the right. The project explorer shows a project named 'spring-security-jwt' with a package structure including 'com.cognizant.jwt', 'com.cognizant.jwt.config', 'com.cognizant.jwt.controller', and 'AuthenticationController.java'. The code editor displays the 'AuthenticationController.java' file, which contains the following code:

```
36 Map<String, String> map = new HashMap<>();
37
38 map.put("token", token);
39
40 LOGGER.info("END");
41
42 return map;
43 }
44
45 private String getUser(String authHeader) {
46
47     LOGGER.debug("Authorization Header : {}", authHeader);
48
49     String encodedCredentials = authHeader.substring("Basic ".length());
50
51     byte[] decodedBytes =
52         Base64.getDecoder().decode(encodedCredentials);
53
54     String credentials = new String(decodedBytes);
55
56     LOGGER.debug("Decoded Credentials : {}", credentials);
57
58     return credentials.substring(0, credentials.indexOf(":"));
59 }
60
61 private String generateJwt(String user) {
62
63     JwtBuilder builder = Jwts.builder();
64
65     builder.setSubject(user);
66
67     builder.setIssuedAt(new Date());
68 }
```

Generate token based on the user

Steps to generate token:

- Include JWT library by including the following maven dependency.



- Create a new method in `AuthenticationController` with below method signature:

```

7
8 @RestController
9 public class AuthenticationController {
10
11     private static final Logger LOGGER =
12         LoggerFactory.getLogger(AuthenticationController.class);
13
14     @GetMapping("/authenticate")
15     public Map<String, String> authenticate(
16         @RequestHeader("Authorization") String authHeader) {
17
18         LOGGER.info("START");
19
20         LOGGER.debug(authHeader);
21
22         String user = getUser(authHeader);
23
24         String token = generateJwt(user);
25
26         Map<String, String> map = new HashMap<>();
27
28         map.put("token", token);
29
30         LOGGER.info("END");
31
32         return map;
33     }
34

```

- Generate the token based on the code specified below.

```

JwtBuilder builder = Jwts.builder();
builder.setSubject(user);

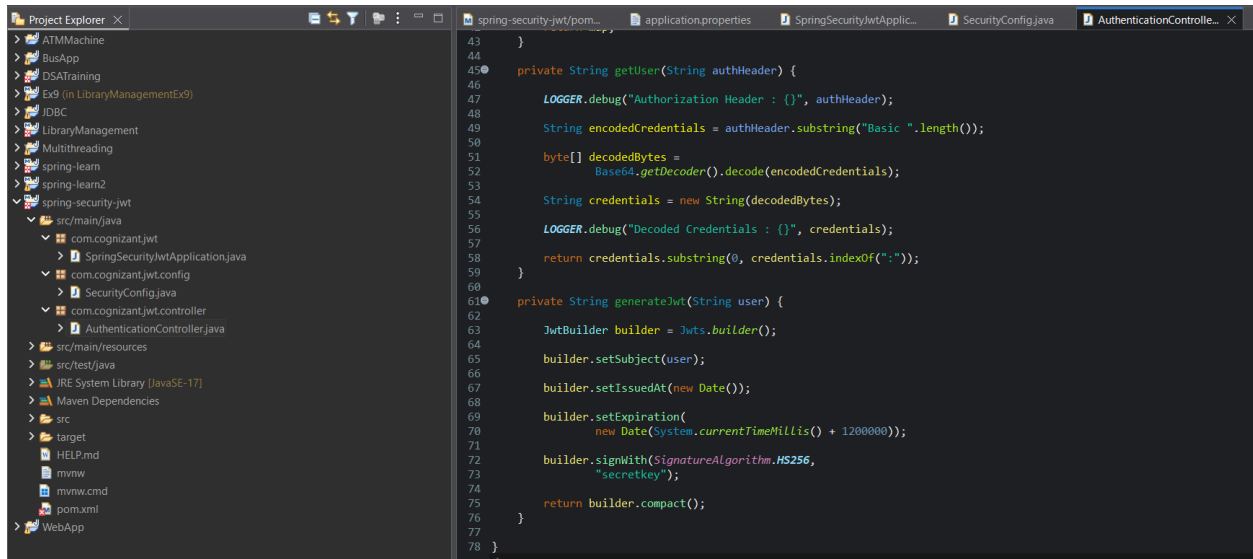
// Set the token issue time as current time
builder.setIssuedAt(new Date());

// Set the token expiry as 20 minutes from now
builder.setExpiration(new Date((new Date()).getTime() + 1200000));
builder.signWith(SignatureAlgorithm.HS256, "secretkey");

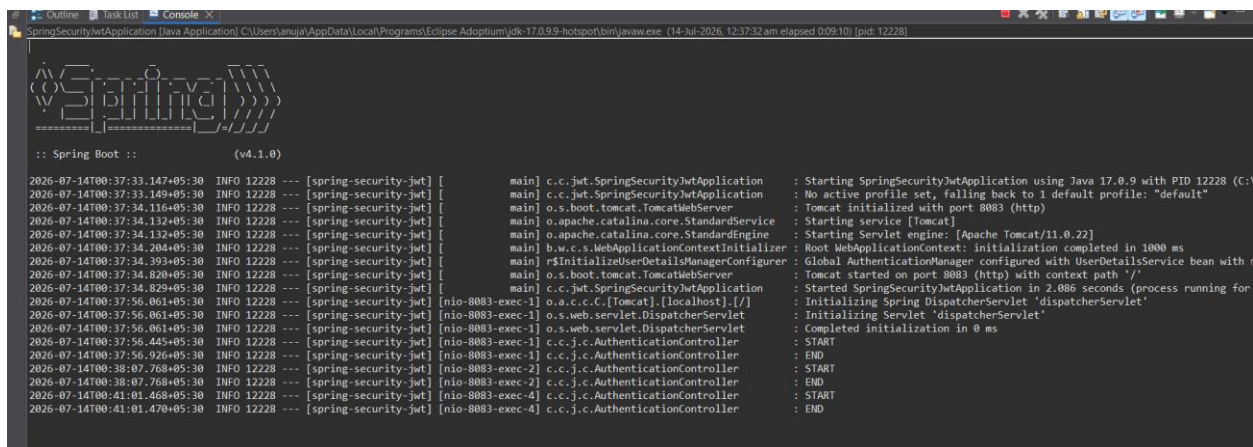
String token = builder.compact();

return token;

```



Running the app:



Output after authentication:

